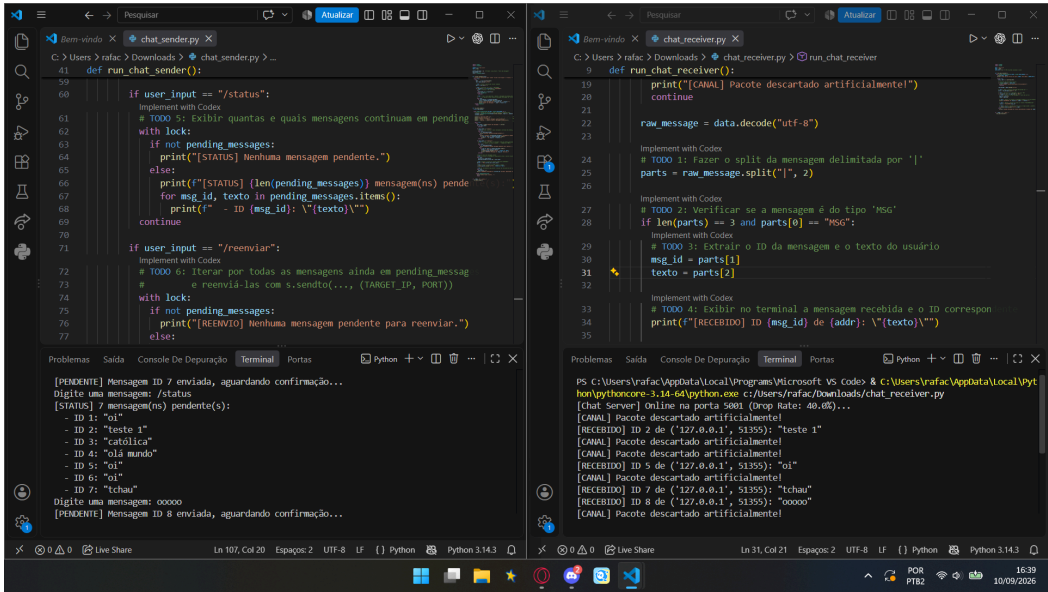


Toda mensagem enviada pelo cliente ganha um ID próprio e é armazenada no dicionário pending messages, ficando marcada como pendente até que sua entrega seja confirmada. Quando a confirmação chega, a mensagem correspondente é removida dessa lista, indicando que ela foi recebida com sucesso pelo servidor.

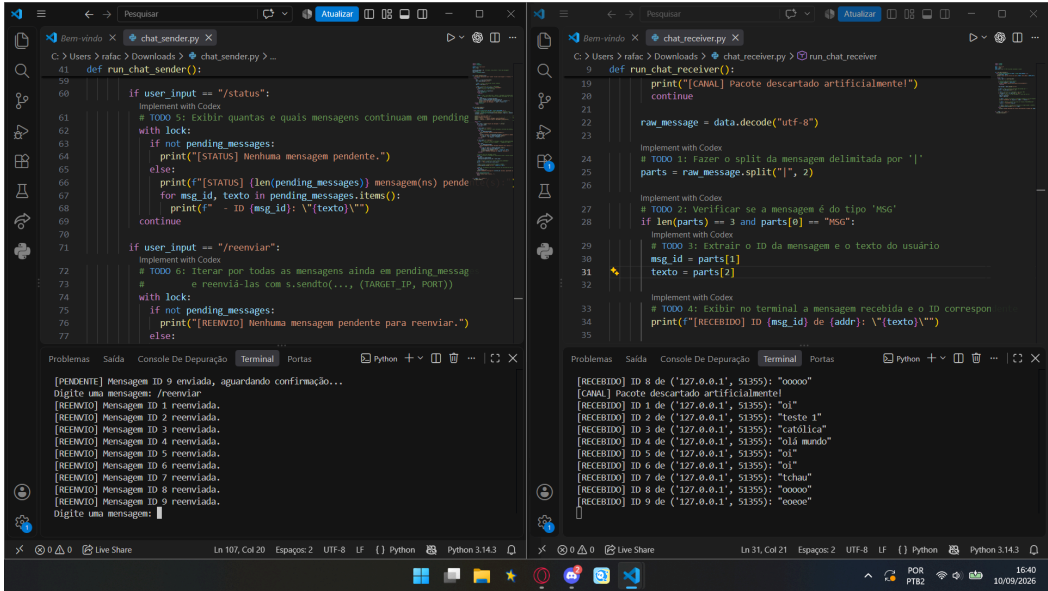
Porém, como o canal de comunicação simula uma rede instável (com 40% de chance de perda), pode acontecer de a mensagem não chegar ao destinatário. Nesses casos, o servidor exibe um aviso de **descarte de pacote**, e como nenhum recibo é enviado de volta, a mensagem permanece pendente na lista — aguardando ser reenviada posteriormente.



The screenshot shows two VS Code windows. The left window displays `chat_sender.py` with a terminal showing the sender's logic and output. The right window displays `chat_receiver.py` with a terminal showing the receiver's logic and output, including several "Pacote descartado artificialmente!" messages.

```
def run_chat_sender():
    if user_input == "/status":
        # TODO 5: Exibir quantas e quais mensagens continuam em pending
        with lock:
            if not pending_messages:
                print("[STATUS] Nenhuma mensagem pendente.")
            else:
                print(f"[STATUS] {len(pending_messages)} mensagem(ns) pendente(s) para reenvio.")
                for msg_id, texto in pending_messages.items():
                    print(f"ID {msg_id}: {texto}")
        continue
    if user_input == "/reenviar":
        # TODO 6: Iterar por todas as mensagens ainda em pending_message
        # e reenvia-las com s.sendto(..., (TARGET_IP, PORT))
        with lock:
            if not pending_messages:
                print("[REENVIO] Nenhuma mensagem pendente para reenviar.")
            else:
                for msg_id, texto in pending_messages.items():
                    s.sendto(texto.encode(), (TARGET_IP, PORT))
                pending_messages.clear()
        continue
```

```
def run_chat_receiver():
    print("[CANAL] Pacote descartado artificialmente!")
    continue
    raw_message = data.decode("utf-8")
    # TODO 1: Fazer o split da mensagem delimitada por '|'
    parts = raw_message.split("|", 2)
    # TODO 2: Verificar se a mensagem é do tipo 'MSG'
    if len(parts) == 3 and parts[0] == "MSG":
        # TODO 3: Extrair o ID da mensagem e o texto do usuário
        msg_id = parts[1]
        texto = parts[2]
        # TODO 4: Exibir no terminal a mensagem recebida e o ID correspondente
        print(f"[RECEBIDO] ID {msg_id} de {addr}: '{texto}'")
```



The screenshot shows the same VS Code windows as above, but with updated terminal output. The sender's terminal shows the retransmission of messages, and the receiver's terminal shows the successful receipt of messages, including the one that was previously discarded.

```
def run_chat_sender():
    if user_input == "/status":
        # TODO 5: Exibir quantas e quais mensagens continuam em pending
        with lock:
            if not pending_messages:
                print("[STATUS] Nenhuma mensagem pendente.")
            else:
                print(f"[STATUS] {len(pending_messages)} mensagem(ns) pendente(s) para reenvio.")
                for msg_id, texto in pending_messages.items():
                    print(f"ID {msg_id}: {texto}")
        continue
    if user_input == "/reenviar":
        # TODO 6: Iterar por todas as mensagens ainda em pending_message
        # e reenvia-las com s.sendto(..., (TARGET_IP, PORT))
        with lock:
            if not pending_messages:
                print("[REENVIO] Nenhuma mensagem pendente para reenviar.")
            else:
                for msg_id, texto in pending_messages.items():
                    s.sendto(texto.encode(), (TARGET_IP, PORT))
                pending_messages.clear()
        continue
```

```
def run_chat_receiver():
    print("[CANAL] Pacote descartado artificialmente!")
    continue
    raw_message = data.decode("utf-8")
    # TODO 1: Fazer o split da mensagem delimitada por '|'
    parts = raw_message.split("|", 2)
    # TODO 2: Verificar se a mensagem é do tipo 'MSG'
    if len(parts) == 3 and parts[0] == "MSG":
        # TODO 3: Extrair o ID da mensagem e o texto do usuário
        msg_id = parts[1]
        texto = parts[2]
        # TODO 4: Exibir no terminal a mensagem recebida e o ID correspondente
        print(f"[RECEBIDO] ID {msg_id} de {addr}: '{texto}'")
```

Camada de aplicação:

Controle total sobre quando reenviar, visibilidade do estado (/status), flexibilidade para customizar a lógica

Trabalhoso de implementar, sujeito a bugs (race conditions, exige lock), sem otimizações como controle de congestionamento

TCP:

Automático (ACKs, retransmissão, ordem garantida), robusto e otimizado.

Menos controle/flexibilidade para a aplicação, maior latência